LegalSENN Documentation

Self-Explaining Neural Networks for Legal Text Classification

Author: Nikolaos Roufas
Affiliation: Sapienza Università di Roma

Proprietary Codebase
All rights reserved. This software is the property of Nikolaos Roufas and Sapienza University. Redistribution or use without permission is prohibited.

December 2, 2025

Contents

1 Introduction **2**

2 Repository Structure **2**

3 Installation **3**
- 3.1 Environment Creation . . . 3
- 3.2 Install Dependencies . . . 3

4 Dataset Preparation (LexGLUE) **3**
- 4.1 SCOTUS Example (Recommended) . . . 3
- 4.2 EURLEX Example . . . 4

5 Model Architecture **4**
- 5.1 1. Transformer Encoder . . . 4
- 5.2 2. SENN Head . . . 4

6 Training **5**
- 6.1 Apple Silicon Optimization . . . 5

7 Inference and Explanation **5**
- 7.1 Output Example . . . 5

8 Explanation of the Codebase **6**
- 8.1 model.py . . . 6
- 8.2 data.py . . . 6
- 8.3 train.py . . . 6
- 8.4 inference.py . . . 7

9 Git Usage and Version Control **7**
- 9.1 Git Initialization . . . 7
- 9.2 Set Main Branch . . . 7
- 9.3 Link GitHub Repository . . . 7
- 9.4 Push . . . 7

10 Citations **8**
- 10.1 Self-Explaining Neural Networks . . . 8
- 10.2 LexGLUE . . . 8

1 Introduction

LegalSENN is an interpretable deep learning architecture designed for legal text classification. It integrates a pretrained Transformer encoder (such as Legal-BERT) with a Self-Explaining Neural Network (SENN) head, enabling prediction accompanied by concept-level explanations.

The system is built to operate on datasets from the LexGLUE benchmark, including SCOTUS, EURLEX, LEDGAR, and ECHR-related tasks. It supports end-to-end processing:

1. Dataset preparation from LexGLUE.
2. Transformer-based document encoding.
3. SENN concept extraction and relevance computation.
4. Training, evaluation, and inference.
5. Explanation generation for user-provided trials or judgments.

This documentation describes installation, dataset creation, training, model architecture, inference, and system design.

2 Repository Structure

```
Self-Explainable-Neural-Networks/

          model.py
          data.py
          train.py
          inference.py
          prepare_lexglue_csv.py
          requirements.txt
          README.md
          venv/
          data/
              lexglue_csv/
```

Each file serves a modular role:

- **model.py** — LegalSENN architecture.

- **data.py** — Dataset loader and label mappings.
- **train.py** — Training loop with MPS/GPU support.
- **inference.py** — User-facing prediction and explanation.
- **prepare_lexglue_csv.py** — Conversion of LexGLUE datasets.

3 Installation

3.1 Environment Creation

```
python3 -m venv venv
source venv/bin/activate
```

3.2 Install Dependencies

```
pip install -r requirements.txt
```

4 Dataset Preparation (LexGLUE)

LegalSENN expects a dataset of the form:

```
text,label
"Full text of legal document", "5"
```

LexGLUE tasks are automatically downloaded and converted to CSV format.

4.1 SCOTUS Example (Recommended)

```
python prepare_lexglue_csv.py \
    --task scotus \
    --output_dir data/lexglue_csv \
    --primary_label_only
```

This produces:

```
data/lexglue_csv/scotus/train.csv
data/lexglue_csv/scotus/val.csv
data/lexglue_csv/scotus/test.csv
```

4.2 EURLEX Example

```
python prepare_lexglue_csv.py \
    --task eurlex \
    --output_dir data/lexglue_csv \
    --primary_label_only
```

Note: Multi-label tasks use only the first label when `--primary_label_only` is enabled.

5 Model Architecture

The LegalSENN architecture consists of two components:

5.1 1. Transformer Encoder

A pretrained language model such as Legal-BERT:

$$z(x) = \text{Encoder}(x)_{[CLS]}$$

5.2 2. SENN Head

Concept encoder:

$$h_i(x) = W_c z(x)$$

Relevance predictor:

$$\alpha_i(x) = \phi(z(x))$$

Concept contributions:

$$c_i(x) = h_i(x) \cdot \alpha_i(x)$$

Final classifier:

$$f(x) = W_g c(x) + b_g$$

This design follows the SENN interpretability framework proposed by Alvarez-Melis & Jaakkola (2018).

6 Training

Train the system with:

```
python train.py \
  --train_csv data/lexglue_csv/scotus/train.csv \
  --val_csv data/lexglue_csv/scotus/val.csv \
  --encoder_name nlpaueb/legal-bert-base-uncased \
  --output_dir outputs/legal_senn_scotus \
  --num_epochs 3 \
  --batch_size 2 \
  --max_length 512 \
  --num_concepts 16
```

6.1 Apple Silicon Optimization

The model automatically uses the MPS backend if available:

```
export PYTORCH_ENABLE_MPS_FALLBACK=1
```

7 Inference and Explanation

To classify a user's trial document:

```
python inference.py \
  --model_dir outputs/legal_senn_scotus \
  --text_file my_trial.txt \
  --top_k 3
```

Or provide text directly:

```
python inference.py \
  --model_dir outputs/legal_senn_scotus \
  --text "Full text of document..." \
  --top_k 3
```

7.1 Output Example

```
=== Top-k predicted labels ===
3        prob=0.7213
5        prob=0.1832
```

```
1       prob=0.0955

=== Concept contributions ===
Concept 03:  +2.1145
Concept 07:  +1.8733
Concept 01:  -0.4542
```

8 Explanation of the Codebase

8.1 model.py

Defines:

- Transformer encoder
- Concept layer
- Relevance MLP
- Contribution computation
- Prediction method

8.2 data.py

Handles:

- Tokenization
- Label mapping
- PyTorch dataset creation

8.3 train.py

Implements:

- DataLoader
- Training loop

- MPS/CUDA CPU fallback
- Best model checkpointing

8.4 inference.py

Responsible for:

- Loading trained model
- Tokenizing user text
- Predicting top-k labels
- Printing SENN concept contributions

9 Git Usage and Version Control

9.1 Git Initialization

```
git init
git add .
git commit -m "Initial commit"
```

9.2 Set Main Branch

```
git branch -M main
```

9.3 Link GitHub Repository

```
git remote add origin https://github.com/<USERNAME>/<REPO>.git
```

9.4 Push

```
git push -u origin main
```

10 Citations

10.1 Self-Explaining Neural Networks

```
@inproceedings{alvarez2018senn,
  title={Towards robust interpretability with self-explaining neural n
  author={Alvarez-Melis, David and Jaakkola, Tommi},
  booktitle={NeurIPS},
  year={2018}
}
```

10.2 LexGLUE

```
@article{chalkidis2022lexglue,
  title={LexGLUE: A benchmark dataset for legal language understanding
  author={Chalkidis, Ilias and others},
  year={2022}
}
```